

Python Fundamentals

30 interview practice questions

Output tracing / debugging / short implementations / explanations

Each numbered question has a distinct primary focus. Related details are tested in its follow-ups. These are practice reconstructions based on your notes.

Assume CPython 3.12 for runnable snippets. Trace printed output; for a failure, name the exception type and where it occurs. The GIL question also asks about newer free-threaded builds.

Predict before running. Explain the rule behind your answer. For an implementation, check both the normal case and the stated edge cases. Solutions are intentionally omitted.

- 01 Context manager: entry, exit, and exception suppression
- 02 Stacked decorators: decoration time versus call time
- 03 `__new__` versus `__init__`: debug a singleton
- 04 `async with`: implement asynchronous resource cleanup
- 05 `async for`: implement an asynchronous iterator
- 06 `typing.Protocol`: structural interfaces
- 07 Parameterized decorator: write a decorator factory
- 08 `functools.wraps`: preserve function metadata
- 09 GIL: reason about CPU-bound threads
- 10 Multithreading versus multiprocessing: choose an execution model
- 11 Lookup complexity: indexing, membership, and hashing
- 12 List comprehension: flatten, filter, and transform
- 13 Dictionary comprehension: duplicate keys
- 14 `map`, `filter`, and `reduce`: functional data processing
- 15 Iterator protocol: `__iter__`, `__next__`, and exhaustion
- 16 Generator: lazy execution and suspended state
- 17 `finally`: cleanup and return precedence
- 18 Exception handlers: broad versus specific exceptions
- 19 Custom exception: validate domain rules
- 20 Classes and objects: class versus instance attributes
- 21 Inheritance: initialize the base class with `super`
- 22 Polymorphism: one operation, interchangeable objects
- 23 Encapsulation: properties and name mangling
- 24 Method overloading versus overriding
- 25 `staticmethod` versus `classmethod`: subclass-aware factories
- 26 Collection types: list, tuple, set, and dictionary
- 27 Mutable versus immutable: mutation and rebinding
- 28 Equality versus identity: `==` and `is`
- 29 Shallow copy versus deep copy: nested references
- 30 Lambda functions: closure capture and late binding

01. Context manager: entry, exit, and exception suppression

PREDICT OUTPUT + EXPLAIN

```
class Session:
    def __enter__(self):
        print("enter")
        return "ready"

    def __exit__(self, exc_type, exc, tb):
        print("exit", exc_type.__name__ if exc_type else None)
        return True

print("before")
with Session() as value:
    print("body", value)
    raise ValueError("bad")
    print("unreachable")
print("after")
```

1. Predict the exact printed output and explain what is assigned to value.
2. What changes if `__exit__` returns False? Does the final print run?
3. If `__enter__` itself raises, is `__exit__` called? Explain how a context manager supports resource cleanup.

02. Stacked decorators: decoration time versus call time

PREDICT OUTPUT + EXPLAIN

```
def outer(func):
    print("apply outer")
    def wrapper():
        print("outer before")
        result = func()
        print("outer after")
        return result
    return wrapper

def inner(func):
    print("apply inner")
    def wrapper():
        print("inner before")
        result = func()
        print("inner after")
        return result
    return wrapper

@outer
@inner
def job():
    print("job")
    return 7

print("call")
print(job())
```

1. Predict every printed line, including lines printed when job is defined.
2. Rewrite the decorator syntax as a single assignment without @.
3. Explain the order of decorator application and the order of wrapper execution.

03. `__new__` versus `__init__`: debug a singleton

DEBUG + EXPLAIN

```
class Connection:
    _instance = None

    def __new__(cls, label):
        print("new", label)
        if cls._instance is None:
            cls._instance = super().__new__(cls)
            # A required statement is missing.

        def __init__(self, label):
            print("init", label)
            self.label = label

a = Connection("first")
b = Connection("second")
print(a is b)
print(a)
```

1. Predict the printed output of this broken program and identify the missing statement.
2. After fixing `__new__`, what would `print(a.label, b.label)` show? Why?
3. Modify the single-threaded implementation so the shared instance is initialized only once.
4. Explain the object-creation lifecycle and give another reason to override `__new__`.

04. async with: implement asynchronous resource cleanup

IMPLEMENT + TRACE

```
import asyncio

class AsyncConnection:
    def __init__(self):
        self.open = False

    async def __aenter__(self):
        pass # Implement.

    async def __aexit__(self, exc_type, exc, tb):
        pass # Implement.

async def main():
    try:
        async with AsyncConnection() as conn:
            print(conn.open)
            raise RuntimeError("failure")
    except RuntimeError:
        print("caught")

asyncio.run(main())
```

1. Implement entry: await asyncio.sleep(0), set open to True, print 'opened', and return the connection.
2. Implement exit: await asyncio.sleep(0), set open to False, print 'closed', and allow exceptions to propagate.
3. Predict the completed program's output. Why is async with useful when acquisition or cleanup needs await?

05. async for: implement an asynchronous iterator

IMPLEMENT + EXPLAIN

```
import asyncio

class AsyncRange:
    def __init__(self, stop):
        self.current = 0
        self.stop = stop

    def __aiter__(self):
        pass # Implement.

    async def __anext__(self):
        pass # Implement.

async def main():
    async for value in AsyncRange(3):
        print(value)

asyncio.run(main())
```

1. Implement an iterator yielding integers from 0 up to, but excluding, a non-negative stop.
2. Await `asyncio.sleep(0)` before returning each value; terminate using the appropriate exception.
3. Explain why `__anext__` is asynchronous and whether `__aiter__` should be `async def` here.

06. typing.Protocol: structural interfaces

EXPLAIN + DIAGNOSE

```
from typing import Protocol

class DocumentSource(Protocol):
    def fetch(self, doc_id: str) -> str: ...

class MemorySource:
    def fetch(self, doc_id: str) -> str:
        return f"document:{doc_id}"

class BrokenSource:
    def fetch(self, doc_id: str) -> int:
        return 123

def read_document(source: DocumentSource) -> str:
    return source.fetch("A")

print(read_document(MemorySource()))
```

1. Why can MemorySource satisfy DocumentSource without inheriting from it?
2. Would a static type checker accept BrokenSource? Does the annotation automatically enforce the return type at runtime?
3. What happens with isinstance(MemorySource(), DocumentSource)? How does @runtime_checkable change the check, and what does it fail to validate?
4. Give a situation where a Protocol would help you design interchangeable components.

07. Parameterized decorator: write a decorator factory

IMPLEMENT

```
def announce(prefix):
    pass # Implement the decorator factory.

@announce("TRACE")
def add(a, b=0):
    return a + b

print(add(2, b=3))
```

1. Implement announce so each function call prints '<prefix>:<function name>' before calling the function exactly once.
2. Support any positional and keyword arguments and return the original function's result.
3. Explain the three layers: factory, decorator, and wrapper.

08. functools.wraps: preserve function metadata

PREDICT OUTPUT + FIX

```
def traced(func):
    def wrapper(*args, **kwargs):
        return func(*args, **kwargs)
    return wrapper

@traced
def total(a, b):
    """Add two numbers."""
    return a + b

print(total.__name__)
print(total.__doc__)
```

1. Predict the exact output before making any changes.
2. Add functools.wraps in the correct place and predict the new output.
3. What is `__wrapped__`, and how does preserved metadata help documentation, debugging, or introspection?

09. GIL: reason about CPU-bound threads

EXPLAIN

```
def work(n):
    total = 0
    for i in range(n):
        total += i * i
    return total
```

1. Assume conventional CPython with its GIL enabled. What does the GIL restrict?
2. Would four threads running four equally sized work tasks guarantee a speedup over running those same tasks sequentially? Explain.
3. Why can threads still help with I/O? Can native extensions behave differently from this pure-Python loop?
4. Does the GIL guarantee that an arbitrary multi-step update to shared data is race-free? What changes in a free-threaded CPython build?

10. Multithreading versus multiprocessing: choose an execution model

COMPARE + DESIGN

1. System A downloads 200 documents and spends most of its time waiting for responses. System B performs independent, expensive pure-Python computations on 200 documents. Choose `ThreadPoolExecutor` or `ProcessPoolExecutor` for each, and justify the choice.
2. Compare memory sharing, startup overhead, argument serialization, and communication between workers.
3. If a worker appends to an ordinary list, when does the parent see the change: a thread or a separate process? How would you return results from process workers?

11. Lookup complexity: indexing, membership, and hashing

ANALYZE COMPLEXITY

1. Assume values is a list, lookup is a dictionary, and unique is a set, each containing n items. Assume constant-cost hash and equality operations and a valid list index.
2. Give average-case and worst-case time complexity for: values[i]; target in values; key in lookup; lookup[key]; target in unique; target in lookup.values().
3. Explain why list indexing and list membership differ, and why dictionary/set lookup is described as average $O(1)$.

12. List comprehension: flatten, filter, and transform

PREDICT OUTPUT + REWRITE

```
rows = [[1, 2, 3], [4], [], [5, 6]]
result = [
    n * n
    for row in rows
    for n in row
    if n % 2 == 0
]
print(result)
```

1. Predict the exact output and explain the order of the two for clauses.
2. Rewrite the comprehension as ordinary nested loops.
3. Change the comprehension to square only odd numbers greater than 2, preserving their original traversal order.

13. Dictionary comprehension: duplicate keys

PREDICT OUTPUT + MODIFY

```
pairs = [("A", 2), ("B", 3), ("A", 5)]
result = {key: value * 2 for key, value in pairs}
print(result)
```

1. Predict the exact output. What happens when a key occurs more than once?
2. Modify the comprehension to include only pairs whose value is at least 3, while retaining the multiplication by 2.
3. Rewrite the comprehension as an ordinary loop, then describe how you would retain all repeated-key values as lists instead.

14. map, filter, and reduce: functional data processing

PREDICT OUTPUT + REWRITE

```
from functools import reduce

values = [1, 2, 3, 4, 5]
mapped = map(lambda n: n * 2, values)
selected = filter(lambda n: n > 5, mapped)

print(reduce(lambda total, n: total + n, selected, 0))
print(list(mapped))
print(list(selected))
```

1. Predict the exact output and explain transformation, selection, and aggregation separately.
2. Which operations create lazy iterators, and which operation consumes them here?
3. Rewrite the whole transformation/filtering/summing pipeline with an ordinary loop.
4. What happens when reduce receives an empty iterable with, and without, an initializer?

15. Iterator protocol: __iter__, __next__, and exhaustion

PREDICT OUTPUT + EXPLAIN

```
class Countdown:
    def __init__(self, start):
        self.current = start

    def __iter__(self):
        return self

    def __next__(self):
        if self.current <= 0:
            raise StopIteration
        value = self.current
        self.current -= 1
        return value

counter = Countdown(3)
print(next(counter))
print(list(counter))
print(list(counter))
```

1. Predict the exact output.
2. Explain the difference between an iterable and an iterator. Is an ordinary list itself an iterator?
3. Why does this object not restart on a second traversal? Describe a design in which __iter__ returns a fresh iterator.

16. Generator: lazy execution and suspended state

PREDICT OUTPUT + EXPLAIN

```
def scores():
    print("start")
    yield 10
    print("middle")
    yield 20
    print("end")

g = scores()
print("created")
print(next(g))
print(list(g))
print(list(g))
```

1. Predict every printed line and identify when the function body begins executing.
2. Explain how yield differs from return and how the function resumes after a yield.
3. How is a generator related to an iterator? Why might a generator help when processing a very large file?

17. finally: cleanup and return precedence

PREDICT OUTPUT + EXPLAIN

```
def evaluate():
    try:
        print("try")
        return "from try"
    finally:
        print("finally")
        return "from finally"

print(evaluate())
```

1. Predict the exact output.
2. What changes if the return statement inside finally is removed?
3. Under normal interpreter execution, explain when finally runs after successful completion, return, or an exception.
4. If try raises an exception but finally returns a value, what happens to the exception?

18. Exception handlers: broad versus specific exceptions

PREDICT OUTPUT + FIX

```
values = ["10", "bad", None]

for value in values:
    try:
        print(int(value))
    except Exception:
        print("general")
    except ValueError:
        print("invalid number")
    except TypeError:
        print("unsupported type")
```

1. Predict the exact output and explain why some handlers never run.
2. Reorder the handlers so conversion and unsupported-type errors receive their intended messages.
3. Does `except Exception` catch `KeyboardInterrupt` or `SystemExit`? Explain the relevant exception hierarchy.

19. Custom exception: validate domain rules

IMPLEMENT

1. Define `InvalidScoreError` as a subclass of `ValueError`.
2. Implement `validate_score(value)`: accept integers from 0 through 100 inclusive, reject booleans explicitly, raise `InvalidScoreError` with a useful message for every invalid input, and return the accepted score.
3. Write a caller that processes `[50, True, -1, 101, '90']`, catches `InvalidScoreError` for each rejected item, and continues to the next item.
4. Explain why a custom exception helps callers distinguish this domain failure from other errors.

20. Classes and objects: class versus instance attributes

PREDICT OUTPUT + FIX

```
class Player:
    tags = []

    def __init__(self, name):
        self.name = name

a = Player("A")
b = Player("B")
a.tags.append("winner")
b.name = "C"

print(a.name, b.name)
print(a.tags, b.tags)
```

1. Predict the exact output.
2. Explain why the names are independent but the tags are shared.
3. Fix the class so every player has its own tags list. Explain the roles of the class, each object, self, and __init__.

21. Inheritance: initialize the base class with super

DEBUG + EXPLAIN

```
class User:
    def __init__(self, name):
        self.name = name
        print("User init")

class Admin(User):
    def __init__(self, name, level):
        self.level = level
        print("Admin init")

admin = Admin("Jay", 2)
print(admin.name, admin.level)
```

1. State what is printed before the program fails, and identify the exception type.
2. Fix Admin so it initializes both inherited and subclass attributes; call the base initializer first.
3. Predict the corrected output and explain why the base initializer was not automatically run in the original code.

22. Polymorphism: one operation, interchangeable objects

PREDICT OUTPUT + EXPLAIN

```
class EmailSender:
    def send(self, message):
        return f"email:{message}"

class SmsSender:
    def send(self, message):
        return f"sms:{message}"

def notify(sender, message):
    return sender.send(message)

for sender in [EmailSender(), SmsSender()]:
    print(notify(sender, "hello"))
```

1. Predict the exact output and explain why notify works with both unrelated classes.
2. Add a PushSender without changing notify.
3. Does this form of runtime polymorphism require a common base class? What happens if an object lacks send?

23. Encapsulation: properties and name mangling

PREDICT OUTPUT + IMPLEMENT

```
class Account:
    def __init__(self):
        self.__balance = 100

    @property
    def balance(self):
        return self.__balance

account = Account()
account.__balance = 999
print(account.balance)
print(account.__balance)
```

1. Predict the exact output. Did the external assignment modify the attribute used by the property?
2. Explain double-underscore name mangling and whether it provides strict privacy.
3. What happens if you execute `account.balance = -50`?
4. Add a `withdraw(amount)` method that rejects non-positive amounts and overdrafts, while preserving a non-negative balance. Assume `amount` is an integer.

24. Method overloading versus overriding

DEBUG + COMPARE

```
class Formatter:
    def format(self, value):
        return f"one:{value}"

    def format(self, value, prefix):
        return f"{prefix}:{value}"

formatter = Formatter()
print(formatter.format("hello"))
```

1. Identify the exception and explain why two declarations did not create two runtime overloads.
2. Replace them with one method that supports both `format('hello')` and `format('hello', 'tag')`.
3. Explain how overriding a method in a subclass differs from this attempted overloading.
4. Does `typing.overload` alone implement runtime argument dispatch?

25. staticmethod versus classmethod: subclass-aware factories

PREDICT OUTPUT + EXPLAIN

```
class Player:
    def __init__(self, name):
        self.name = name

    @classmethod
    def from_record(cls, record):
        return cls(record["name"])

    @staticmethod
    def valid_name(name):
        return bool(name.strip())

class VipPlayer(Player):
    pass

player = VipPlayer.from_record({"name": "Jay"})
print(type(player).__name__, player.name)
print(VipPlayer.valid_name(" "))
```

1. Predict the exact output.
2. Explain what is passed automatically to an instance method, a classmethod, and a staticmethod.
3. What changes if `from_record` returns `Player(record['name'])` instead of `cls(record['name'])`?
4. Why is a classmethod useful for an alternative constructor?

26. Collection types: list, tuple, set, and dictionary

PREDICT OUTPUT + COMPARE

```
values = [3, 1, 3, 2]
as_tuple = tuple(values)
as_set = set(values)
as_dict = {value: value * 10 for value in values}

print(len(values), len(as_tuple), len(as_set), len(as_dict))
print(list(as_dict))
```

1. Predict the exact output; do not assume any iteration order for a set.
2. Compare the four types on ordering, duplicates, positional indexing, and mutability.
3. Choose a type for each: an ordered editable task list; a fixed coordinate pair; unique player IDs; player-name lookup by ID.
4. What requirement must a set element or dictionary key satisfy?

27. Mutable versus immutable: mutation and rebinding

PREDICT OUTPUT + EXPLAIN

```
def change(items, count):
    items += [3]
    count += 1
    return count

numbers = [1, 2]
count = 0
returned = change(numbers, count)
print(numbers, count, returned)

pair = ([10], 20)
pair[0].append(30)
print(pair)
```

1. Predict the exact output.
2. Explain why the caller's list changes but the caller's integer binding does not.
3. Why can a list inside a tuple change? What happens with `pair[0] = [99]`?

28. Equality versus identity: == and is

PREDICT OUTPUT + EXPLAIN

```
a = [1, 2]
b = [1, 2]
c = a

print(a == b, a is b)
print(a == c, a is c)
c.append(3)
print(a == b, a is c)
```

1. Predict the exact output and explain each comparison.
2. Which operator should compare collection contents, and which should check for None?
3. Why should code not rely on is to compare ordinary strings or integers by value?

29. Shallow copy versus deep copy: nested references

PREDICT OUTPUT + EXPLAIN

```
import copy

original = {"scores": [10, 20]}
shallow = copy.copy(original)
deep = copy.deepcopy(original)

shallow["scores"].append(30)
shallow["label"] = "updated"
deep["scores"].append(40)

print(original)
print(shallow)
print(deep)
```

1. Predict the exact output.
2. Explain which dictionaries and nested lists are shared.
3. Why does adding label to shallow not add it to original, while appending to shallow['scores'] affects original?
4. How would plain assignment differ from both copy operations?

30. Lambda functions: closure capture and late binding

PREDICT OUTPUT + FIX

```
functions = [lambda: i * 2 for i in range(3)]  
print([function() for function in functions])
```

1. Predict the exact output and explain when the variable `i` is read.
2. Fix the lambdas so they remember the value of `i` from their own creation and produce the intended doubled values.
3. Explain what a lambda function is, its expression-versus-statement restriction, and one useful application as a key function.

Reference baseline

Official Python documentation checked for language protocols, decorator metadata, structural typing, concurrency, and container complexity. Questions and examples are original practice material.

Data model: [object creation, iterators, and asynchronous protocols](#)

Compound statements: [with, try, and function decoration](#)

[functools](#): [wraps](#) and [reduce](#)

[typing](#): [Protocol](#) and [overload](#)

[threading](#): [GIL](#) and [free-threaded builds](#)

[Python Wiki](#): [CPython container time complexity](#)